

5. 度量学习与对比学习

- DINO
- JEPA, I-JEPA, V-JEPA

1. 背景与动机

在机器学习的传统范式中，我们主要关注的是分类和回归问题，即给定输入样本预测其类别标签或数值输出。然而，在许多实际应用中，我们更关心的是不同样本之间的相似性或关系。无论是在最早的最近邻分类、聚类分析，还是在现代的信息检索、推荐系统、计算机视觉与自然语言处理任务中，很多模型的性能往往依赖于对“相似”与“不相似”的合理刻画。**度量学习 (Metric Learning)** 正是为了解决这类问题而产生的一个重要研究领域：通过学习一种合适的距离度量或嵌入表示，使得相似样本在该空间中距离更近，而不相似样本更远。

早期的度量学习多以线性方法为主，例如基于马氏距离的学习方法，着力于通过凸优化手段直接构造合适的度量函数。随着深度学习的兴起，研究者逐渐将深度神经网络引入到度量学习框架中，由此形成了**深度度量学习 (Deep Metric Learning)**。这一阶段的核心思想是利用深度网络自动提取特征，并结合对比性约束（如三元组损失、对比损失）来优化表示空间，使得复杂、高维的数据能够更好地体现语义相似关系。

与此同时，另一条研究路径也在迅速发展——**对比学习 (Contrastive Learning)**。它起源于深度度量学习中的损失函数设计，但逐渐发展为一种更为通用的表征学习范式。在无监督与自监督学习的背景下，对比学习不再仅仅依赖明确的“标签对”，而是通过构造“正样本对”和“负样本对”的对比关系，让模型自动学习到具有判别性的表示。典型方法如SimCLR、MoCo等，不仅推动了图像表征学习的突破，也在自然语言处理与多模态任务中取得了广泛成功。

因此，度量学习与对比学习之间存在紧密的继承关系：**度量学习提供了关于“相似性建模”的基本思想，对比学习则将这一思想推广到更大规模、更弱监督甚至无监督的场景**。可以说，对比学习是深度度量学习在自监督学习范式下的自然延伸与升华。

本部从经典的度量学习方法出发，逐步过渡到深度度量学习，再引入对比学习的基本原理，并结合典型应用与案例进行讲解。

2. 经典度量学习方法

2.1 度量学习的概念

度量学习从数据中学习一个距离度量函数，使得该函数能够准确地反映样本间的相似性关系。距离度量 (Distance Metric) 是定义在空间上的一个函数 $d: \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}^+$ ，对于任意的样本 $\mathbf{x}^{(i)}, \mathbf{x}^{(j)}, \mathbf{x}^{(k)} \in \mathcal{X}$ ，该函数需要满足四个基本性质：

- 非负性： $d(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) \geq 0$ ，即距离总是非负的。
- 同一性： $d(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = 0$ 当且仅当 $\mathbf{x}^{(i)} = \mathbf{x}^{(j)}$ ，即只有相同的样本之间距离才为零。

- 对称性： $d(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = d(\mathbf{x}^{(j)}, \mathbf{x}^{(i)})$ ，即距离函数关于两个参数是对称的。
- 三角不等式： $d(\mathbf{x}^{(i)}, \mathbf{x}^{(k)}) \leq d(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) + d(\mathbf{x}^{(j)}, \mathbf{x}^{(k)})$ ，即任意两点间的直接距离不超过通过第三点的间接距离。

相似度量（Similarity Metric）则是距离度量的对偶概念，通常定义为 $s: \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ ，其值越大表示两个样本越相似。相似度和距离之间存在多种转换关系。一种常见方法的是：

$$s(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \exp(-d(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}))$$

当相似度在0到1之间时也常使用：

$$d(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = 1 - s(\mathbf{x}^{(i)}, \mathbf{x}^{(j)})$$

传统的机器学习方法通常使用预定义的距离度量，如欧氏距离：

$$d_{\text{Euclidean}}(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \|\mathbf{x}^{(i)} - \mathbf{x}^{(j)}\|_2$$

或曼哈顿距离：

$$d_{\text{Manhattan}}(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \|\mathbf{x}^{(i)} - \mathbf{x}^{(j)}\|_1$$

然而，这些通用的距离度量往往无法很好地刻画特定任务中数据的内在结构和相似性关系。

度量学习的核心思想就是“学习何为相似”。给定一个训练数据集，其中包含样本之间的相似性信息（如样本对的相似/不相似标签，或者样本的类别标签），度量学习算法的目标是学习一个参数化的距离度量函数 $d_{\theta}(\mathbf{x}^{(i)}, \mathbf{x}^{(j)})$ ，使得该函数在训练数据上能够准确地反映样本间的相似性关系，并且能够泛化到新的样本对上。

更具体地，我们可以将度量学习形式化为以下优化问题：

$$\theta^* = \arg \min_{\theta} \sum_{(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) \in \mathcal{S}} L(d_{\theta}(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}), y^{(ij)}) + \lambda R(\theta)$$

其中 $\mathcal{S}$ 是训练样本对的集合， $y^{(ij)}$ 表示样本对 $(\mathbf{x}^{(i)}, \mathbf{x}^{(j)})$ 的相似性标签， $L(\cdot, \cdot)$ 是损失函数， $R(\theta)$ 是正则化项， λ 是正则化参数。

度量学习的最终目标是学习一个能够准确反映数据内在相似性结构的距离度量，使得在该度量下相似的样本聚集在一起，不相似的样本被分离开来。这种学习到的度量可以直接用于各种下游任务，如检索、聚类、分类等，从而提高这些任务的性能。

2.2 Mahalanobis距离与线性度量学习

线性度量学习的核心思想是通过学习一个线性变换矩阵，将原始特征空间映射到一个新的空间，在该空间中使用欧氏距离能够更好地反映样本间的相似性关系。

Mahalanobis距离是线性度量学习的理论基础，它是欧氏距离的推广形式。给定两个样本 $\mathbf{x}^{(i)}, \mathbf{x}^{(j)} \in \mathbb{R}^d$ ，它们之间的Mahalanobis距离定义为：

$$d_{\mathbf{M}}(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \sqrt{(\mathbf{x}^{(i)} - \mathbf{x}^{(j)})^{\top} \mathbf{M} (\mathbf{x}^{(i)} - \mathbf{x}^{(j)})}$$

其中 $\mathbf{M} \in \mathbb{R}^{d \times d}$ 称为度量矩阵，通常为对称矩阵。当 $\mathbf{M}$ 为单位矩阵时，Mahalanobis距离退化为欧氏距离。

为了保证距离函数的非负性，矩阵 $\mathbf{M}$ 必须是半正定的。进一步地，为了满足距离度量的同一性要求（即 $d(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = 0$ 当且仅当 $\mathbf{x}^{(i)} = \mathbf{x}^{(j)}$ ），矩阵 $\mathbf{M}$ 通常要求是正定的。

我们可以将正定矩阵 $\mathbf{M}$ 分解为 $\mathbf{M} = \mathbf{L}^\top \mathbf{L}$ 的形式，其中 $\mathbf{L} \in \mathbb{R}^{k \times d}$ （ $k \leq d$ ）。这样，Mahalanobis距离可以重写为：

$$d_{\mathbf{M}}(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \|\mathbf{L}(\mathbf{x}^{(i)} - \mathbf{x}^{(j)})\|_2$$

该式揭示了Mahalanobis距离的几何意义，它等价于先对样本进行线性变换，然后在变换后的空间中计算欧氏距离。矩阵 $\mathbf{L}$ 可以理解为一个特征变换矩阵，它将原始特征映射到一个更适合度量相似性的新特征空间。

线性度量学习的目标就是从训练数据中学习合适的度量矩阵 $\mathbf{M}$ 或变换矩阵 $\mathbf{L}$ 。假设我们有一个训练集 $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^N$ ，其中 $y^{(i)}$ 是样本 $\mathbf{x}^{(i)}$ 的类别标签。我们可以构造正样本对集合 $\mathcal{S}^+ = \{(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) : y^{(i)} = y^{(j)}\}$ 和负样本对集合 $\mathcal{S}^- = \{(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) : y^{(i)} \neq y^{(j)}\}$ 。

最简单的线性度量学习目标是使同类样本间的距离尽可能小，异类样本间的距离尽可能大：

$$\min_{\mathbf{M} \succeq 0} \sum_{(\mathbf{x}_i, \mathbf{x}_j) \in \mathcal{S}^+} d_{\mathbf{M}}^2(\mathbf{x}_i, \mathbf{x}_j) - \sum_{(\mathbf{x}_i, \mathbf{x}_j) \in \mathcal{S}^-} d_{\mathbf{M}}^2(\mathbf{x}_i, \mathbf{x}_j) + \lambda \|\mathbf{M}\|_F^2$$

其中 $\mathbf{M}$ 是半正定矩阵， $\|\mathbf{M}\|_F^2$ 是Frobenius范数（矩阵元素平方和开方）的平方，作为正则化项防止过拟合。

2.3 *大间隔最近邻方法

大间隔最近邻（Large Margin Nearest Neighbor, LMNN）是最具代表性的线性度量学习算法之一，其核心思想是学习一个度量，使得在该度量下每个样本的 k 个最近邻都属于同一类别，同时来自不同类别的样本被一个较大的间隔分离开。

LMNN算法的设计受到支持向量机的启发，它将最近邻分类的思想与大间隔学习相结合。具体来说，对于每个样本 $\mathbf{x}^{(i)}$ ，LMNN首先在原始欧氏距离下找到它的 k 个同类最近邻，称为**目标邻居**（target neighbors）。然后学习一个新的度量，使得在该度量下这些目标邻居仍然是 $\mathbf{x}^{(i)}$ 的最近邻，同时确保来自不同类别的样本（称为入侵者，imposters）与 $\mathbf{x}^{(i)}$ 的距离足够远。

令 $\eta^{(ij)} = 1$ 表示样本 $\mathbf{x}^{(j)}$ 是样本 $\mathbf{x}^{(i)}$ 的目标邻居，否则 $\eta^{(ij)} = 0$ 。LMNN的目标函数包含两个部分：

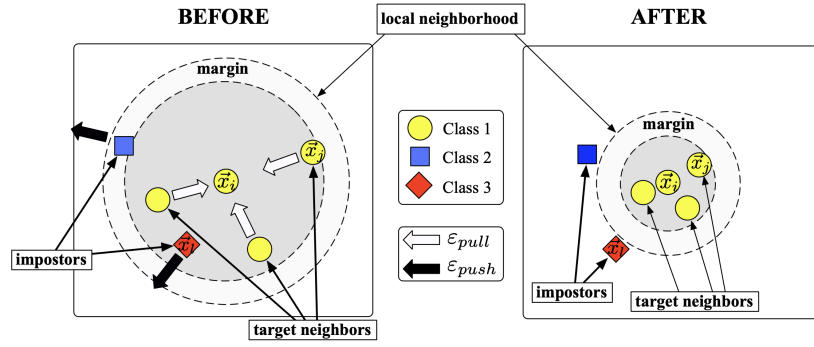
$$\min_{\mathbf{M}} (1 - \mu) \sum_{i,j} \eta^{(ij)} d_{\mathbf{M}}^2(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) + \mu \sum_{i,j,l} \eta^{(ij)} (1 - y^{(il)}) \xi^{(ijl)}$$

第一项称为拉力项（pull term），它使每个样本与其目标邻居的距离尽可能小。第二项是推力项（push term），它通过引入松弛变量 $\xi^{(ijl)}$ 来确保入侵者 $\mathbf{x}^{(l)}$ （满足 $y^{(il)} = 0$ ，即与 $\mathbf{x}^{(i)}$ 类别不同）与 $\mathbf{x}^{(i)}$ 的距离比目标邻居 $\mathbf{x}^{(j)}$ 与 $\mathbf{x}^{(i)}$ 的距离大一个间隔。参数 $\mu \in [0, 1]$ 平衡两项的重要性。

模型约束条件为：

$$\begin{aligned} d_{\mathbf{M}}^2(\mathbf{x}^{(i)}, \mathbf{x}^{(l)}) - d_{\mathbf{M}}^2(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) &\geq 1 - \xi^{(ijl)} \\ \xi^{(ijl)} &\geq 0 \end{aligned}$$

该约束确保了对于每个样本 $\mathbf{x}^{(i)}$ 和它的目标邻居 $\mathbf{x}^{(j)}$ ，任何异类样本 $\mathbf{x}^{(l)}$ 到 $\mathbf{x}^{(i)}$ 的距离都应该比 $\mathbf{x}^{(j)}$ 到 $\mathbf{x}^{(i)}$ 的距离至少大1个单位（减去松弛量 $\xi^{(ijl)}$ ）。



LMNN优化问题是一个半定规划（Semidefinite Programming, SDP）问题，可以使用专门的SDP求解器来求解。然而，由于约束条件的数量可能非常大（ $O(kn^3)$ ，其中 k 是邻居数量， n 是样本数量），直接求解在大规模数据集上是不现实的。

为了提高计算效率，研究者提出了多种近似求解方法。一种常用的方法是使用次梯度下降（subgradient descent）来优化目标函数。在每次迭代中，算法随机选择一个三元组 $(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}, \mathbf{x}^{(l)})$ ，其中 $\mathbf{x}^{(j)}$ 是 $\mathbf{x}^{(i)}$ 的目标邻居， $\mathbf{x}^{(l)}$ 是违反间隔约束的入侵者，然后对度量矩阵 $\mathbf{M}$ 进行更新。

LMNN算法的另一个重要特点是它可以自然地处理多类分类问题，不需要将其分解为多个二分类问题。此外，学习到的度量不仅可以用于 k 近邻分类，还可以用于其他需要相似性度量的任务，如聚类、检索等。

2.4 *非线性度量学习

虽然线性度量学习在许多应用中都取得了成功，但它的表达能力受到线性变换的限制。在实际数据中，样本间的相似性关系往往是非线性的，这时就需要考虑非线性度量学习方法。

由于度量学习模型关注的是样本之间的距离，因此很容易利用核技巧将线性度量学习模型扩展为非线性度量学习模型。令 $\kappa(\cdot, \cdot)$ 为核函数，核化的度量学习目标是学习一个核矩阵的线性组合：

$$\kappa_{\beta}(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \sum_{l=1}^N \beta_l \kappa(\mathbf{x}^{(l)}, \mathbf{x}^{(i)}) \kappa(\mathbf{x}^{(l)}, \mathbf{x}^{(j)})$$

其中 $\beta = [\beta_1, \beta_2, \dots, \beta_n]^T$ 是需要学习的参数向量。

相应的高维映射空间中的距离定义为：

$$d_{\beta}^2(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \kappa_{\beta}(\mathbf{x}^{(i)}, \mathbf{x}^{(i)}) + \kappa_{\beta}(\mathbf{x}^{(j)}, \mathbf{x}^{(j)}) - 2\kappa_{\beta}(\mathbf{x}^{(i)}, \mathbf{x}^{(j)})$$

非线性度量学习的优化问题的形式为：

$$\min_{\beta} \sum_{(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) \in \mathcal{S}^+} d_{\beta}^2(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) - \sum_{(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) \in \mathcal{S}^-} d_{\beta}^2(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) + \lambda \|\beta\|^2$$

其中正则化项 $\|\beta\|^2$ 防止过拟合。

3. 深度度量学习

深度度量学习结合了深度学习强大的特征表示能力和度量学习的相似性建模思想，从而能够捕捉更加全面的数据特征，适应复杂多变的应用场景。

3.1 模型基本组成

深度度量学习的基本思想是使用深度神经网络同时学习特征表示和相似性度量。模型主要包括嵌入网络（embedding network）和距离度量函数两个组件。嵌入网络表示为 $f_{\theta} : \mathbb{R}^d \rightarrow \mathbb{R}^k$ ，其作用是将原始输入映射到一个 k 维嵌入空间；距离度量函数用于在嵌入空间中计算样本间的相似性。

给定原始样本 $\mathbf{x}^{(i)} \in \mathbb{R}^d$ ，嵌入网络将其映射为嵌入向量 $\mathbf{z}^{(i)} = f_{\theta}(\mathbf{x}^{(i)}) \in \mathbb{R}^k$ 。然后在嵌入空间中定义距离度量，最常用的是欧氏距离：

$$d(\mathbf{x}_i, \mathbf{x}_j) = \|\mathbf{z}_i - \mathbf{z}_j\|_2 = \|f_{\theta}(\mathbf{x}_i) - f_{\theta}(\mathbf{x}_j)\|_2$$

深度度量学习的目标是通过优化损失函数来同时学习网络参数 θ 和相应的度量性质。这种端到端的学习方式使得特征表示能够针对具体的相似性任务进行优化，从而获得更好的性能。

3.2 特征空间的几何性质

深度度量学习通过非线性神经网络学习嵌入，把样本从复杂的原始空间映射到一个新的特征空间。在这个空间中，利用简单的线性度量（如欧氏距离、余弦相似度）就能有效度量样本间的语义相似性。此外，通过将高维嵌入向量投影到二维或三维空间，我们甚至可以真接通过可视化的方式观察不同类别的样本在嵌入空间中的分布情况。

理想的度量学习结果应该呈现以下几何性质：

- **类内紧致性（Intra-class Compactness）**：同一类别的样本在嵌入空间中应该聚集在一起，形成紧致的簇。这可以通过计算类内方差来量化：

$$\text{Var}_{\text{intra}} = \frac{1}{C} \sum_{c=1}^C \frac{1}{|S_c|} \sum_{\mathbf{x}^{(i)} \in S_c} \|\mathbf{z}^{(i)} - \boldsymbol{\mu}_c\|^2$$

其中 C 是类别数量， S_c 是第 c 类的样本集合， $\boldsymbol{\mu}_c$ 是第 c 类样本在嵌入空间中的均值。

- **类间分离性（Inter-class Separability）**：不同类别的样本在嵌入空间中应该被明显分离开来。这可以通过计算类间距离来量化：

$$\text{Dist}_{\text{inter}} = \frac{1}{C(C-1)} \sum_{c=1}^C \sum_{c' \neq c} \|\boldsymbol{\mu}_c - \boldsymbol{\mu}_{c'}\|^2$$

- **均匀分布性（Uniform Distribution）**：为了最大化嵌入空间的利用效率，不同类别的样本应该在嵌入空间中均匀分布，避免过度集中在某个局部区域。

3.3 Siamese Network

Siamese Network（孪生网络）是深度度量学习中最经典的网络架构之一。它由两个或多个共享参数的子网络组成，每个子网络处理一个输入样本，然后通过某种方式比较子网络的输出来计算样本间的相似性。

给定一对样本 $(\mathbf{x}_i, \mathbf{x}_j)$ ，两个共享参数的子网络 f_{θ} 分别计算它们的嵌入表示：

$$\mathbf{z}^{(i)} = f_{\theta}(\mathbf{x}^{(i)}), \quad \mathbf{z}^{(j)} = f_{\theta}(\mathbf{x}^{(j)})$$

然后使用某种距离函数（如欧氏距离、余弦距离等）计算嵌入向量间的距离：

$$d(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \|\mathbf{z}^{(i)} - \mathbf{z}^{(j)}\|_2$$

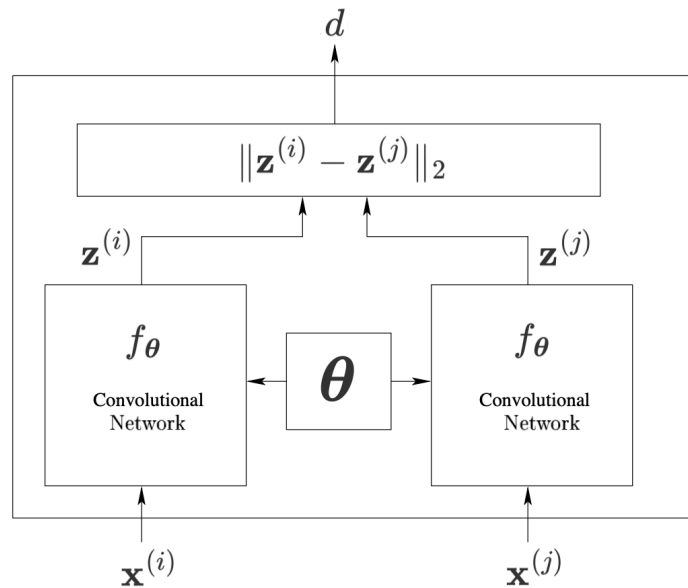
Siamese Network使用对比损失（Contrastive Loss）进行训练：

$$L_{\text{contrastive}} = \sum_{(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}, y^{(ij)})} y^{(ij)} \cdot d^2(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) + (1 - y^{(ij)}) \cdot \max(0, \tau - d(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}))^2$$

其中 $y^{(ij)} \in \{0, 1\}$ 是样本对的标签（1表示相似，0表示不相似）， τ 是间隔参数。

对比损失的设计思想是：对于相似的样本对，最小化它们的距离；对于不相似的样本对，当它们的距离小于间隔 τ 时，增大它们的距离，当距离已经大于等于 τ 时，不再施加额外的惩罚。

Siamese Network 参数共享减少了模型复杂度，训练相对简单，可以处理变长的输入序列。不过，其缺点是只能处理样本对，无法直接利用多个样本间的关系信息。



3.4 Triplet Network

Triplet Network（三元组网络）是对Siamese Network的扩展，它同时处理三个样本：一个锚点样本（anchor） $\mathbf{x}_a$ ，一个正样本（positive） $\mathbf{x}_p$ （与锚点相似），和一个负样本（negative） $\mathbf{x}_n$ （与锚点不相似）。

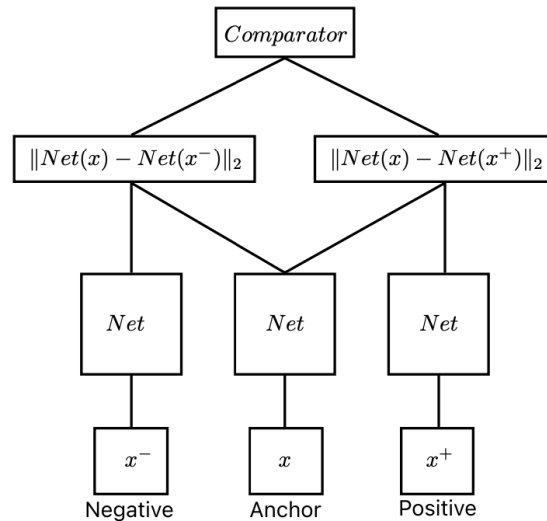
Triplet Network的结构包含三个共享参数的子网络：

$$\mathbf{z}_a = f_{\theta}(\mathbf{x}_a), \quad \mathbf{z}_p = f_{\theta}(\mathbf{x}_p), \quad \mathbf{z}_n = f_{\theta}(\mathbf{x}_n)$$

Triplet Network使用三元组损失（Triplet Loss）进行训练：

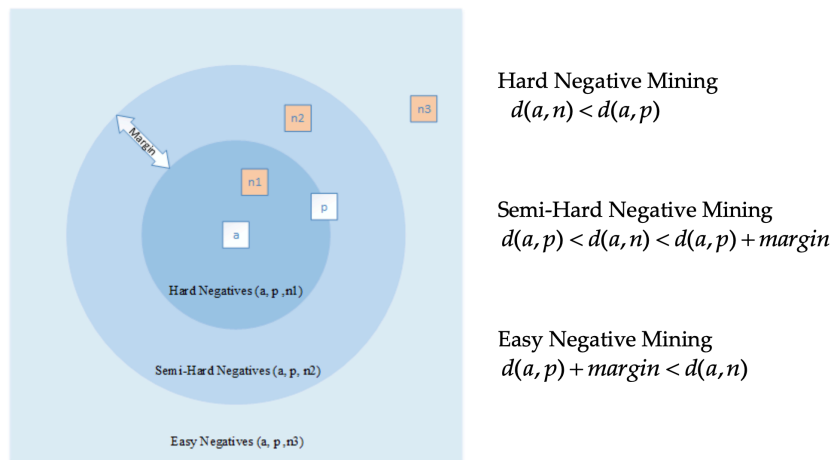
$$L_{\text{triplet}} = \sum_{(\mathbf{x}_a, \mathbf{x}_p, \mathbf{x}_n)} \max(0, d(\mathbf{x}_a, \mathbf{x}_p) - d(\mathbf{x}_a, \mathbf{x}_n) + \tau)$$

三元组损失的核心思想是确保锚点与正样本的距离比锚点与负样本的距离小至少 τ 个单位。这种相对比较的方式比绝对的距离约束更加鲁棒，能够更好地学习样本间的相对关系。



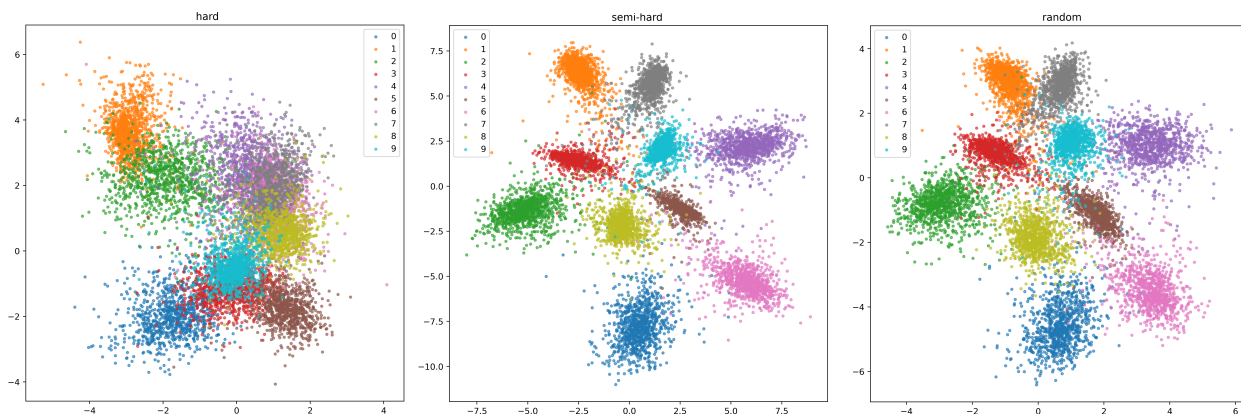
由于可能的三元组数量是 $O(n^3)$ ，直接使用所有三元组既不现实也不必要，选择能够提供最大信息量的三元组是提高模型训练效率和性能的关键技术，称为**三元组挖掘**。常用的三元组挖掘策略包括：

- **Hard Negative Mining**：选择那些距离锚点最近的负样本，即 $\mathbf{x}_n = \arg \min_{\mathbf{x}_k \in \mathcal{N}_a} d(\mathbf{x}_a, \mathbf{x}_k)$ ，其中 $\mathcal{N}_a$ 是锚点 $\mathbf{x}_a$ 的负样本集合。
- **Semi-Hard Negative Mining**：选择那些距离锚点比正样本远，但仍然在间隔内的负样本：
 $d(\mathbf{x}_a, \mathbf{x}_p) < d(\mathbf{x}_a, \mathbf{x}_n) < d(\mathbf{x}_a, \mathbf{x}_p) + \tau$
- **Random Sampling**：随机选择三元组，虽然效率较低，但可以避免陷入局部最优。



在实际训练中，通常采用在线挖掘（online mining）的策略，即在每个mini-batch内动态选择难例三元组，这样既保证了训练效率，又能充分利用难例信息。

Triplet Network相比Siamese Network的优势在于能够更直接地建模相对关系，训练更加稳定。但它需要更复杂的三元组采样策略，训练时间也相对较长。



4. 对比学习基础

对比学习（Contrastive Learning）是近年来机器学习领域最具影响力的发展之一，特别是在自监督表征学习方面取得了突破性的进展。对比学习与度量学习密切相关，但其关注点不在“度量”，而是更多地集中在无监督或自监督的表征学习上。

4.1 对比学习的基本原理

与度量学习通过比较两个或多个样本来建模相似性的思路不同，对比学习更强调样本**自身的表示**。更确切地说，它通常在一个样本与其经过数据增强或变换后的版本之间建立对比关系。由于对比学习并不直接依赖显式的“度量”目标，因此需要通过设计 **代理任务（pretext tasks）** 来提供监督信号。不同的对比学习方法往往对应着不同的代理任务，从而引导模型学习判别性表示。

对比学习的核心思想可以追溯到信息论中的互信息最大化原理。给定数据 $\mathbf{x}$ 和其变换版本 $\mathbf{x}^+$ （称为正样本），对比学习的目标是学习一个编码器 f_θ ，使得 $f_\theta(\mathbf{x})$ 和 $f_\theta(\mathbf{x}^+)$ 之间的互信息最大化。

形式化地，对比学习要解决的优化问题可以表述为：

$$\max_{\theta} I(f_{\theta}(\mathbf{x}); f_{\theta}(\mathbf{x}^+))$$

其中 $I(\cdot; \cdot)$ 表示互信息。直接计算互信息在实际中通常是不可行的，对比学习框架通过代理任务来解决这个问题。

对比学习的成功很大程度上依赖于如何构造有效的正样本对和负样本对。正样本对应该在语义上相似，而负样本对应该在语义上不相似。

正样本对的构造策略包括：

- 在计算机视觉中，常用的数据增强技术包括随机裁剪、颜色抖动、高斯模糊、水平翻转等。给定原始图像 $\mathbf{x}$ ，通过应用随机的数据增强组合 $t \sim T$ 和 $t' \sim T'$ ，可以生成正样本对 $(t(\mathbf{x}), t'(\mathbf{x}))$ 。这些变换应该保持图像的语义内容不变，但改变其表面特征。
- 在自然语言处理中，正样本对可以通过同义词替换、句子重写、回译等方法生成。例如，“这部电影很精彩”和“这部影片非常棒”可以构成一个正样本对。

负样本对的构造通常更加直接：

- 最简单的方法是将batch内的其他样本作为负样本。对于样本 $\mathbf{x}_i$ ，batch内的其他所有样本 $\{\mathbf{x}_j\}_{j \neq i}$ 都可以作为

负样本。这种方法简单有效，但假设batch内的样本都不相似，这在实际中可能不成立。

- 更复杂的负样本构造方法包括使用内存库（memory bank）存储历史的负样本，或者使用生成模型生成对抗样本作为负样本。

4.2 InfoNCE损失函数

度量学习中常用的对比损失、三元组损失也常用于对比学习之中。但是，对比学习中最重要的是损失函数是InfoNCE（Information Noise-Contrastive Estimation），它为对比学习提供了坚实的理论基础。

给定一个查询样本 $\mathbf{x}$ 、一个正样本 $\mathbf{x}^+$ 和 N 个负样本 $\{\mathbf{x}_i^-\}_{i=1}^N$ ，InfoNCE损失定义为：

$$L_{\text{InfoNCE}} = -\log \frac{\exp(f_{\theta}(\mathbf{x})^{\top} f_{\theta}(\mathbf{x}^+)/\tau)}{\exp(f_{\theta}(\mathbf{x})^{\top} f_{\theta}(\mathbf{x}^+)/\tau) + \sum_{i=1}^N \exp(f_{\theta}(\mathbf{x})^{\top} f_{\theta}(\mathbf{x}_i^-)/\tau)}$$

其中 τ 是温度参数，用于控制分布的锐化程度：

- 当 $\tau \rightarrow 0$ 时，损失函数变得越来越尖锐，只关注最相似的样本
- 当 $\tau \rightarrow \infty$ 时，损失函数变得平坦，所有样本被同等对待
- 合适的 τ 值通常在0.1到1.0之间，需要通过实验确定

InfoNCE损失可以理解为一个 $(N+1)$ 类分类问题，其中正样本是正确类别， N 个负样本是错误类别。这种理解使得我们可以使用交叉熵损失的优化技巧。

理论分析表明，在某些条件下，最小化InfoNCE损失等价于最大化查询样本和正样本之间的互信息的下界：

$$I(f_{\theta}(\mathbf{x}); f_{\theta}(\mathbf{x}^+)) \geq \log(N+1) - L_{\text{InfoNCE}}$$

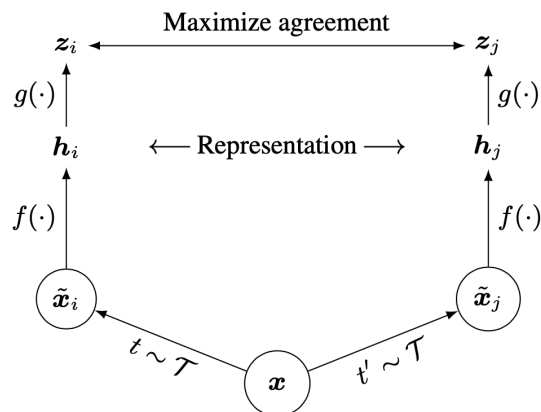
这个结果说明了为什么InfoNCE损失在对比学习中如此有效：它直接优化了我们想要最大化的目标（互信息）的一个可计算的下界。

5. 重要对比学习架构

对比学习近年来在表示学习领域取得了显著进展，其核心思想是通过拉近相似样本、推远不相似样本来学习有效的特征表示。本部分介绍四种具有代表性的对比学习架构：SimCLR、MoCo、BYOL 和 SimSiam。

5.1 SimCLR

SimCLR（A Simple Framework for Contrastive Learning of Visual Representations）是对比学习在视觉表示学习中的一项基础性工作，证明了无需手工设计标签，仅通过对比学习即可在大规模图像数据集上取得与监督学习相当甚至更优的性能。其核心思想在于通过数据增强构建正样本对，利用对比损失在一个批次中区分正负样本，从而学习具有高度区分性的表示。



SimCLR架构包含四个关键组件：

- **数据增强模块**：对输入图像 $\mathbf{x}$ 随机应用两种不同的增强操作，包括随机裁剪、颜色抖动、高斯模糊等变换，得到正样本对 $(\mathbf{x}_i, \mathbf{x}_j)$ 。
- **编码器网络**：通常采用 ResNet 等深度卷积神经网络 $f(\cdot)$ 来提取特征表示 $\mathbf{h} = f(\mathbf{x})$ ，负责将原始图像映射到高维特征空间。
- **投影头**：使用一个多层感知机 $g(\cdot)$ 将编码器输出的特征 $\mathbf{h}$ 进一步映射到专门用于对比学习的表示空间 $\mathbf{z} = g(\mathbf{h})$ 。这个看似简单的投影头结构能够显著提升最终的表示质量，是SimCLR成功的关键因素之一。
- **对比损失函数**：采用基于温度参数的InfoNCE损失来拉近正样本对的距离，同时推远负样本对的距离。

对于一个大小为 N 的批次，通过数据增强操作可以得到 $2N$ 个样本。在训练过程中，对于每个样本，来自同一原始图像的另一个增强样本被视为其正样本，而其余 $2N - 2$ 个样本则作为负样本参与对比学习。这种设计使得模型能够在一个批次内同时处理大量的正负样本对，从而提高训练效率。损失函数采用InfoNCE的形式：

$$\mathcal{L} = \frac{1}{2N} \sum_{k=1}^{2N} -\log \frac{\exp(\text{sim}(\mathbf{z}_k, \mathbf{z}_{k^+})/\tau)}{\sum_{j=1}^{2N} \mathbb{1}_{j \neq k} \exp(\text{sim}(\mathbf{z}_k, \mathbf{z}_j)/\tau)}$$

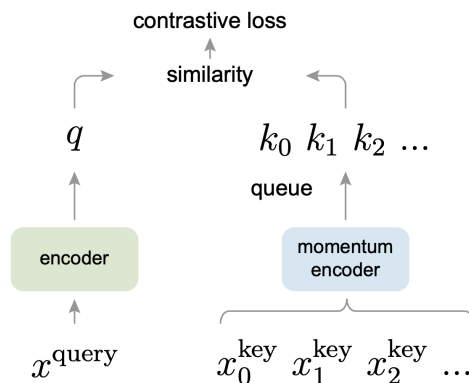
其中 $\text{sim}(\cdot)$ 为余弦相似度函数， τ 为温度超参数，用于控制分布的尖锐程度。

SimCLR有如下几个特点：

- 数据增强策略的组合对最终性能产生显著影响，其中颜色抖动与随机裁剪被证明是最为重要的两种变换方式。
- 投影头的引入可以有效改善表示空间的结构，使得学到的特征更适合下游任务。
- 使用更大的批次大小和更长的训练周期能够显著提升模型性能，这主要归因于更多负样本的参与使得对比学习更加充分。
- 在实际应用中，下游任务通常丢弃投影头部分，仅使用编码器的输出特征进行微调或直接应用。

5.2 MoCo

MoCo (Momentum Contrast) 通过引入动量编码器和队列机制，解决了对比学习中负样本数量受限的关键问题，提升了表示一致性和训练效率。其核心思想在于维护一个动态更新的负样本队列，并通过动量更新机制来保持键编码器的参数稳定性。这种设计允许模型在不显著增加计算开销的前提下实现大规模负样本对比。与传统对比学习方法相比，MoCo能够利用历史批次中的样本作为负样本，大大扩展了对比学习的样本规模。



MoCo的核心组件包括：

- **查询编码器**：使用参数 θ_q 来编码当前批次的查询样本， $q = f_{\theta_q}(x^{\text{query}})$ ，该编码器通过标准的梯度下降方法进行参数更新，确保能够及时学习到最新的特征表示。
- **键编码器**：使用参数 θ_k 来编码键样本， $k = g_{\theta_k}(x_i^{\text{key}})$ ，但与查询编码器不同的是，键编码器采用动量更新策略（ m 为动量系数）：

$$\theta_k \leftarrow m \cdot \theta_k + (1 - m) \cdot \theta_q$$

- **负样本队列**：负责存储历史批次中的键表征，实现大批量负样本的高效复用，避免重复计算的开销。

MoCo架构中，对于给定的查询样本表示 q 和对应的正键样本表示 k^+ ，系统需要在包含大量负样本的候选集合中进行对比学习。损失函数采用对比学习的标准形式，具体定义为：

$$\mathcal{L}_q = -\log \frac{\exp(q \cdot k^+ / \tau)}{\exp(q \cdot k^+ / \tau) + \sum_{i=1}^K \exp(q \cdot k_i / \tau)}$$

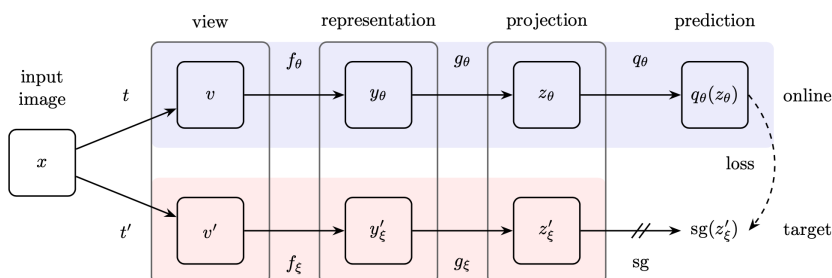
其中 k_i 表示来自队列中的负样本表示， K 为队列大小， τ 为温度参数。

MoCo的主要特点包括：

- 较大的队列带来更多负样本。
- 较高动量（如 $m \geq 0.99$ ）有助于稳定键表示。
- 不依赖超大 batch 即可获得良好结果。

5.3 BYOL

BYOL（Bootstrap Your Own Latent）构建了一种无需显式负样本的自监督框架。它由两个结构相近但参数更新方式截然不同的神经网络（通过非对称网络结构），即在线网络与目标网络，通过最小化二者输出之间的回归损失来实现表示学习，摆脱了传统对比学习对显式负采样的依赖。



BYOL架构的关键组件包括：

- **在线网络**，包括编码器、投影头和预测器三个子模块，所有参数通过标准的梯度下降方法进行更新。编码器负责提取输入图像的基础特征表示，投影头将这些特征映射到适合对比学习的空间，而预测器则负责预测目标网络的输出。
- **目标网络**，在结构上与在线网络保持一致，但其参数更新采用动量机制，为在线网络提供学习目标。

BYOL的损失函数设计为最小化在线网络输出与目标网络输出之间的均方误差：

$$\mathcal{L}_{\text{BYOL}} = \|\bar{q}_{\theta}(\mathbf{z}_{\theta}) - \text{sg}(\mathbf{z}_{\xi})\|^2$$

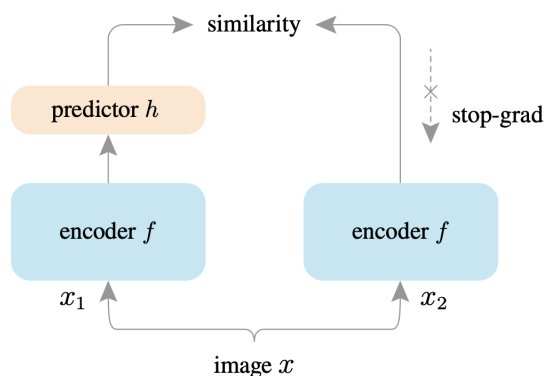
其中 sg 表示停止梯度操作，这是防止模式坍塌的关键技术手段， $\bar{\cdot}$ 表示经过L2标准化后的向量。停止梯度操作确保了目标网络的输出不会直接影响在线网络的梯度计算，从而维持了两个网络之间的非对称性。

BYOL主要特点包括：

- 无需负样本即可实现有效的表示学习
- 预测器模块与动量更新机制是避免模式坍塌的两个关键因素，缺少任何一个都可能导致模型陷入平凡解。
- 批归一化层可能在BYOL中发挥了隐式提供负样本效应的作用，这一定程度上也解释了为什么该方法能够在没有显式负样本的情况下避免模式坍塌。

5.4 SimSiam

SimSiam（Simple Siamese Representation Learning）进一步简化了自监督学习的框架设计，在不使用显式负样本、动量编码器或大批次的情况下，通过孪生结构、预测器与停止梯度实现表示学习。



Algorithm 1 SimSiam Pseudocode, PyTorch-like

```
# f: backbone + projection mlp
# h: prediction mlp

for x in loader: # load a minibatch x with n samples
    x1, x2 = aug(x), aug(x) # random augmentation
    z1, z2 = f(x1), f(x2) # projections, n-by-d
    p1, p2 = h(z1), h(z2) # predictions, n-by-d

    L = D(p1, z2)/2 + D(p2, z1)/2 # loss

    L.backward() # back-propagate
    update(f, h) # SGD update

def D(p, z): # negative cosine similarity
    z = z.detach() # stop gradient

    p = normalize(p, dim=1) # l2-normalize
    z = normalize(z, dim=1) # l2-normalize
    return -(p*z).sum(dim=1).mean()
```

该架构由三个核心组件构成：

- **编码器**：共享权重的孪生编码器 f 和投影头 g 。
- **预测器**：是一个多层感知机 h ，仅应用于网络的一侧分支。
- **停止梯度操作**：在防止模型通过平凡解坍塌为常数输出方面发挥着决定性作用，它确保网络的两个分支在梯度流方面保持适当的独立性。

SimSiam的损失函数采用对称的形式，定义为：

$$\mathcal{L} = \frac{1}{2} [D(\mathbf{p}_1, \text{sg}(\mathbf{z}_2)) + D(\mathbf{p}_2, \text{sg}(\mathbf{z}_1))]$$

其中 D 为余弦距离函数， $\mathbf{p} = h(g(f(\mathbf{x})))$ 表示经过预测器处理的特征， $\mathbf{z} = g(f(\mathbf{x}))$ 表示投影后的特征。这种对称设计确保了训练过程的稳定性，同时停止梯度操作的应用防止了网络退化为平凡解。

SimSiam揭示了自监督学习中的几个关键机制和重要发现：

- 停止梯度操作被确认为避免模型坍塌的核心技术手段，移除这一操作会导致模型迅速收敛到平凡解，所有输出变为常数。
- 预测器的网络结构对最终性能具有重要影响，其深度、宽度以及激活函数的选择都会显著影响学习效果。
- 即使不使用动量编码器或负样本，SimSiam仍能在ImageNet等具有挑战性的基准数据集上取得与当时先进方法相当的竞争性结果。

5.5 比较分析

本节回顾了四种常见框架：SimCLR 提供端到端对比学习范式；MoCo 通过动量字典缓解大 batch 依赖；BYOL 与 SimSiam 探索了无需显式负样本的学习机制。

方法	使用负样本	需要大 batch	动量编码器	使用预测器	主要特点
SimCLR	是	是	否	否	端到端对比学习；投影头有助于线性评估
MoCo	是	否	是	否	动量更新 + 队列（动态字典）；小批次也可用大量负样本
BYOL	否	否	是	是	在线/目标网络 + EMA；回归式目标，停梯度
SimSiam	否	否	否	是	孪生结构 + 单侧预测器；停梯度避免坍塌

尽管这些方法在形式上有所不同，但其本质都是通过构建某种形式的“目标”来引导表示学习。无论是负样本、动量网络还是预测器，均可视为一种正则化手段，防止模型陷入平凡解并提升泛化能力。

6. 应用

6.1 人脸验证（LFW 数据集）

- **任务**：给定两张人脸，判断是否为同一个人（验证，而非识别）。
- **数据**：LFW（Labeled Faces in the Wild），13,000+ 人脸图像（[宜网](#) 免费下载）。
- **方法**：
 - 经典：用欧氏距离或余弦相似度比较人脸特征（如 PCA/深度特征）。
 - 深度度量学习：训练孪生网络（Siamese Network）或三元组网络（Triplet Network），学习嵌入空间。
- **对比点**：展示深度度量学习如何显著提升验证准确率。

6.4 文本相似度（STS-B 数据集）

- **任务**：判断两句话的语义相似度。
- **数据**：STS-B (Semantic Textual Similarity Benchmark)，包含 7,000+ 对英文句子，每对句子带有相似度分数

(0-5)。

- **方法：**
 - 基线：句子平均词向量 + 余弦相似度。
 - 改进：用对比学习训练句子嵌入（SimCSE, 2021）。
- **对比点：**展示对比学习表征在语义相似度上的提升。

6.3 医学影像检索（MedMNIST）

- **任务：**医学影像分类/检索（例如寻找相似病灶）。
- **数据：**MedMNIST（基于多种公开医学影像数据，[官网](#) 免费获取）。
- **方法：**
 - 基线：直接 CNN 分类或欧氏距离检索。
 - 改进：用对比学习表征，提升小样本任务的性能。
- **对比点：**展示对比学习在小样本医疗场景的优势。

参考文献

- **LMNN:** Distance Metric Learning for Large Margin Nearest Neighbor Classification, 2009.
- **Siamese:** Learning a similarity metric discriminatively, with application to face verification, 2005.
- **Triplet:** Deep Metric Learning Using Triplet Network, 2015.
- **SimCLR:** A Simple Framework for Contrastive Learning of Visual Representations, 2020.
- **MoCo:** Momentum Contrast for Unsupervised Visual Representation Learning, 2020.
- **BYOL:** Bootstrap Your Own Latent A New Approach to Self-Supervised Learning, 2020.
- **SimSam:** Exploring Simple Siamese Representation Learning, 2021.